

# RoboPIM: A ReRAM-Based Accelerator for LLM-Based Robotics Applications via Dynamic Task Slicing

Wenjing Xiao<sup>1</sup>, Member, IEEE, Jianyu Wang, Dan Chen<sup>2</sup>, Huize Li<sup>3</sup>, Mohsen Guizani<sup>4</sup>, Fellow, IEEE, Min Chen<sup>5</sup>, Fellow, IEEE, and Thomas Wu<sup>6</sup>

**Abstract**—Large language models (LLMs) have demonstrated significant potential in improving robotics applications through natural language processing capabilities. Matrix multiplication dominates the computational workload in these LLM-based robotics systems, and resistive random access memory (ReRAM) provides notable performance benefits due to its inherent parallelism and energy efficiency. However, matrix multiplications in these applications pose dual heterogeneity challenges, encompassing both structural and scale heterogeneity. Structurally, these applications involve both standard dense matrix operations and block-structured sparse multiplications, where sparsity patterns vary significantly with robot topology. In terms of scale, matrix sizes range from just a few to thousands of dimensions, making large ReRAM crossbars designed for LLM inefficient for small-scale robotics computations. Existing acceleration approaches fail to address this dual heterogeneity, limiting their effectiveness for LLM-based robotics applications. In this work, we propose robotics applications-processing in memory (RoboPIM), the first low-energy ReRAM-based accelerator tailored for LLM-based robotics applications. robotics applications-processing in memory (RoboPIM) adapts to the diverse matrix multiplication requirements across robot platforms and LLM configurations. Our design features a ReRAM architecture with various crossbar sizes and implements a dynamic task slicing scheme that allocates matrix slices to appropriately sized crossbars. To maximize ReRAM's parallel processing capabilities, we develop a two-stage scheduling

strategy that efficiently manages matrix multiplications while enhancing hardware utilization. Comprehensive evaluations demonstrate that RoboPIM achieves  $2.85\times$  and  $6.85\times$  speedup across diverse robotic platforms and LLMs, respectively, outperforming state-of-the-art domain-specific solutions.

**Index Terms**—Large language model (LLM)-based robotics application, low-energy computing, resistive random access memory (ReRAM).

## I. INTRODUCTION

RECENT advancements in robotics have led to significant developments across various applications, including industrial manufacturing [1], [2] and medical assistance [3], [4]. Subsequently, a series of studies have been proposed to improve the intelligence of robotics [5], [6], particularly through the integration of large language models (LLMs) [7], [8], [9]. Benefiting from the unprecedented ability of LLMs to process and generate natural language, LLM-based robots allow to interpret human instructions more effectively and interact with humans in a more intuitive way, thereby revolutionizing robotics applications.

Fig. 1 illustrates the typical workflow of LLM-based robots. The process begins with humans sending requests, which the robots interpret while simultaneously perceiving the environment through real-time sensor data collection: 1) using the LLM's reasoning capabilities, the robots determine appropriate actions based on the given requests; 2). once a feasible route is identified, the robots execute a series of motion planning and control algorithms to efficiently accomplish tasks; and 3) LLM-based robots integrate perception, LLM-driven reasoning, and motion planning into a closed-loop system, effectively bridging the gap between abstract human intentions and physical robotic actions. However, both LLM inference and motion planning demand intensive computational resources, posing a significant challenge to their practical deployment. To solve the problem, many research projects on accelerating LLMs or robotics applications are emerging. neural processing unit-processing in memory (NeuPIM) [10] and integrated accelerator based on NPU-PIM unified memory system (IANUS) [11] employ heterogeneous architecture to accelerate LLMs by combining neural processing unit (NPU) and DRAM-based processing-in-memory (PIM). Tartan [12] accelerates robotics applications by developing a CPU-based architecture. RoboShape [13] and MOPED [14], [15] propose customized architectures to accelerate multiple robotics appli-

Received 14 May 2025; revised 10 October 2025 and 17 December 2025; accepted 25 January 2026. Date of publication 3 February 2026; date of current version 22 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62502101 and Grant 62462002; and in part by the Natural Science Foundation of Guangxi, China, under Grant 2025GXNSFBA069394 and Grant 2025GXNSFAA069958. This article was recommended by Associate Editor Q. Zhu. (Wenjing Xiao and Jianyu Wang contributed equally to this work.) (Corresponding author: Dan Chen.)

Wenjing Xiao and Jianyu Wang are with the School of Computer, Electronics and Information, Guangxi University, Nanning 530004, China, and also with Guangxi Key Laboratory of Multimedia Communications and Network Technology, Nanning 530004, China (e-mail: wenjingx@gxu.edu.cn; jianyuw@st.gxu.edu.cn).

Dan Chen is with the School of Computing, National University of Singapore, Singapore 119077 (e-mail: danchen@nus.edu.sg).

Huize Li is with the Department of Electrical and Computer Engineering, University of Central Florida, Orlando, FL 32819 USA (e-mail: huize.li@ucf.edu).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates (e-mail: mguizani@ieee.org).

Min Chen is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China, and also with Pazhou Laboratory, Guangzhou 510330, China (e-mail: minchen@ieee.org).

Thomas Wu is with the School of Electrical Engineering, Guangxi University, Nanning 530004, China (e-mail: xwu@gxu.edu.cn).

Digital Object Identifier 10.1109/TCAD.2026.3660201

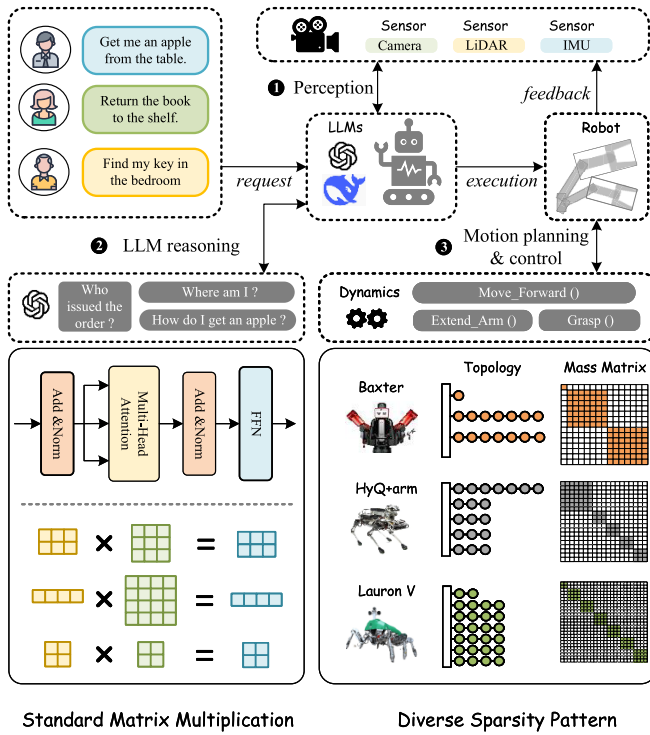


Fig. 1. Illustration of LLM-based robotics application and two main features.

cations such as industrial manipulators and service robots. Despite the promising results of these works, they only focus on accelerating either LLMs or robotics applications, which does not consider to accelerate LLM-based robotics applications in a uniform manner.

In LLM-based robotics applications, the dominant operations are standard or block-diagonal matrix multiplications, which account for the majority of computational time and energy consumption [10], [11], [13]. Thus, a natural insight is: can we design an integrated architecture adapting to various matrix multiplications to accelerate LLM-based robotics applications? Recent works [16], [17], [18] have demonstrated that resistive random access memory (ReRAM) is highly efficient on accelerating matrix-vector multiplication (MVM). Besides, it offers low energy consumption, high storage density, and powerful parallel capabilities [19]. These features make ReRAM particularly well-suited for LLM-based robotics applications. However, since robotics applications and LLMs involve diverse matrix multiplication operations, it is difficult to design a ReRAM-based accelerator for LLM-based robotics applications, due to the following dual heterogeneity.

- 1) *Structural Heterogeneity*: In addition to normal matrix multiplication, sparsity in robotics applications exhibits a block-diagonal pattern and varies significantly with robot topology. This sparsity stems from the inherent correlations and independencies within robot structure. Failing to leverage this sparsity leads to unnecessary calculations and inefficiencies.
- 2) *Scale Heterogeneity*: Matrix sizes vary dramatically, from the small matrices in robot dynamics (tens of dimensions) to the large-scale matrices in LLMs (thou-

sands of dimensions). When the small-scale robotic tasks directly execute on the large crossbars built for LLMs, it leads to severe hardware underutilization and energy inefficiency. Hence, considering this dual heterogeneity is crucial for enhancing processing efficiency in an integrated ReRAM-based architecture.

To this end, we propose the first ReRAM-based accelerator for LLM-based robotics applications, robotics applications-processing in memory (RoboPIM). It is a novel architecture that considers dual heterogeneity of matrix multiplications in LLM-based robotics applications. First, we develop a ReRAM-based integrated architecture with different sized crossbars to support various scales of matrix multiplication. Second, we design a dynamic task slicing scheme to enable uniform matrix processing on ReRAM in a hardware-adapted manner. Specifically, to fully exploit the matrix sparsity that varies with robot specifications, we transform block-diagonal sparse matrix multiplication into multiple dense submatrix multiplications by a sparsity to density task slicing (S2D-Slicing). To effectively coordinate such dense matrix multiplication of varying sizes with hardware specifications, a density to crossbar-adapted task slicing (D2C-Slicing) is further proposed, which partitions the standard matrix multiplication task into several subtasks with crossbar-adapted size, improving the efficiency of matrix processing. Third, based on the dynamic task slicing regime, we introduce a two-stage task scheduling strategy to fully exploit the high parallelism capabilities of the integrated architecture and enhance hardware utilization. Our main contributions of this article are as follows.

- 1) We develop the first integrated ReRAM-based architecture, including crossbars of different sizes for LLM-based robotics applications, enabling efficient matrix multiplication processing.
- 2) We devise a dynamic task slicing scheme to handle diverse sparsity patterns and wide-ranging matrix scales, benefiting both LLM-integrated and traditional robotics.
- 3) We propose a two-stage scheduling strategy to fully exploit the high parallelism capabilities of the integrated ReRAM-based architecture, enhancing the hardware utilization.
- 4) We evaluate our accelerator with a wide range of robots and LLMs. Experimental results show that RoboPIM achieves  $2.85\times$  and  $6.85\times$  speedup across diverse robotic platforms and LLMs, respectively, outperforming state-of-the-art domain-specific solutions.

#### Algorithm 1 Dynamics Gradients With Respect to $u = \{q, \dot{q}\}$

**Step 1:**  $v, a, f \leftarrow \text{RNEA}(q, \dot{q}, \ddot{q})$

**Step 2:**  $\partial\tau/\partial u \leftarrow \nabla\text{RNEA}(\dot{q}, v, a, f)$

**Step 3:**  $\partial\ddot{q}/\partial u \leftarrow -M^{-1}\partial\tau/\partial u$

## II. BACKGROUND AND MOTIVATION

### A. Matrix Multiplication in LLM-Based Robotics Applications

1) *Matrix Multiplications in Robot Dynamics*: Dynamics and dynamics gradients are essential calculations for many

motion planning methods of rigid body robots. Algorithm 1 shows a typical example of forward dynamics gradients [20]. Steps 1 and 2 use joint positions, velocities, and accelerations ( $q, \dot{q}, \ddot{q}$ ) as inputs to compute torque gradients with respect to control inputs  $\partial\tau/\partial u$  using both the recursive Newton–Euler algorithm (RNEA) and its derivative ( $\nabla$ RNEA). Their major operations are standard matrix multiplications. Step 3 involves multiplying dense partial derivative matrices by the sparse inverse of the mass matrix [21], which is a sparse-dense matrix multiplication (SpMM). The mass matrix  $M$  characterizes the distribution of mass and inertia across the robot’s joint space, which is inherently determined by the robot’s topology. For example, Fig. 1 illustrates the topology and corresponding mass matrix of the Lauron V robot [22]. The robot consists of a head with two sequentially connected links and six independent limbs, each comprising four sequentially connected links. The topology of Lauron V robot implies that within head or each limb, the links are connected to each other, leading to a dense subblock in the mass matrix. Meanwhile, different heads or limbs are relatively independent of each other, which results in sparse connections between the subblocks of the mass matrix and forms an overall sparse block diagonal structure. Moreover, the inverse of a block diagonal matrix remains a block diagonal matrix, preserving its sparsity pattern, which can be exploited to efficiently compute dynamics gradients.

2) *Matrix Multiplications in LLMs*: In the field of robotics applications, the introduction of LLMs significantly enhances robots’ interactivity and autonomy. As the foundational structure of LLMs, the transformer consists of alternating blocks, including: multihead attention (MHA), residual connection, layer normalization, and feed-forward network (FFN). We elaborate on the MHA and FFN blocks as follows.

MHA consists of query-key-value (QKV) generation, attention weight calculation, and attention output calculation. Equation (1) describes the QKV generation, where  $W^Q$ ,  $W^K$ , and  $W^V \in \mathbb{R}^{d \times d_k}$  are parameter matrices specific to each head. Here,  $d$  represents the dimension of each token’s feature vector and  $d_k$  is the dimension of the QKV vectors for each head.  $X \in \mathbb{R}^{N \times d}$  is the feature matrix of the input tokens, with  $N$  representing the number of input tokens. Equation (2) describes attention weight calculation, where  $\sigma$  is softmax function. Equation (3) depicts the attention output calculation, where  $S \in \mathbb{R}^{N \times N}$  is the attention weight matrix

$$\mathbf{Q} = \mathbf{X} \cdot \mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X} \cdot \mathbf{W}^K, \quad \mathbf{V} = \mathbf{X} \cdot \mathbf{W}^V \quad (1)$$

$$\mathbf{S} = \sigma \left( \frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}} \right) \quad (2)$$

$$\mathbf{H}_{\text{Attn}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{S} \cdot \mathbf{V}. \quad (3)$$

Equation (4) depicts the FFN, which comprises two linear transformations with an intermediate nonlinear activation function.  $W_1$  and  $W_2$  represent parameter matrices,  $\mathbf{H}_{\text{Attn}}$  is the attention output matrix, and  $b_1$  and  $b_2$  denote bias vectors

$$\mathbf{Z} = \mathbf{W}_2 \cdot \phi(\mathbf{H}_{\text{Attn}} \cdot \mathbf{W}_1 + \mathbf{b}_1) + \mathbf{b}_2. \quad (4)$$

In summary, mass matrices related to multiplication in robot dynamics exhibit inherent block-diagonal sparsity that varies with robot topology, presenting a potential for optimization. Additionally, robot dynamics primarily involves multiple

small-scale matrix multiplication limited by their few degrees of freedom in physical space, while LLM inference requires various large-scale matrix multiplication with thousands of dimensions due to high-dimensional embedding spaces, thus exhibiting large disparities in the scale of matrix operations.

### B. ReRAM-Based PIM

PIM [23], [24], [25], [26], [27] represents an innovative computing paradigm that integrates computational capabilities directly into memory cells, effectively addressing the data movement bottleneck in conventional architectures. Among various PIM implementations, ReRAM stands out as a particularly promising memory technology due to its high storage density and excellent energy efficiency. The fundamental structure of a ReRAM cell comprises a metal oxide layer sandwiched between electrodes. This structure allows binary data representation through distinct resistance states, where high and low resistance levels correspond to logic “0” and “1,” respectively.

The main advantage of ReRAM appears in its crossbar configuration, which enables efficient MVM. In this implementation, matrix elements are mapped as conductance values ( $G$ ) within the crossbar array, while input vectors are transformed into analog voltages ( $V$ ) through digital-to-analog conversion. Following Kirchhoff’s current law, each bitline accumulates current  $I_j = \sum_i V_i \cdot G_{i,j}$ , which is proportional to the dot product of the corresponding input vector and matrix column. These accumulated currents, when converted back to digital values, produce the complete MVM result without requiring data movement to external processing units. This in situ, parallel, and analog computation significantly enhances both computational throughput and energy efficiency for matrix operations that are fundamental to many modern computing workloads.

### C. Motivation

Before addressing specific design challenges, we justify the choice of ReRAM over other hardware candidates. FPGAs and ASICs suffer from high data movement [28] during memory-intensive LLM inference, and static/dynamic random access memory (SRAM) PIMs [29], [30] are limited by low density and significant leakage/refresh power, which are prohibitive for energy-constrained robotics. Consequently, ReRAM offers a superior alternative. It uniquely combines high storage density, nonvolatility, and in situ computing efficiency, making it the optimal substrate to satisfy the dual demands of memory-intensive LLM inference and energy-constrained robotics.

To fully leverage the potential of ReRAM in this context, it is crucial to align the hardware architecture with the specific computational patterns. As described in Section II-A, matrix multiplications are the dominant operations in LLM-based robotics applications. We investigate the unique characteristics of these operations, reveal the inefficiencies in handling them, and highlight the need for a new design to address these challenges.

1) *Observation 1*: The varying block-diagonal sparsity patterns result in unnecessary calculations. We investigated

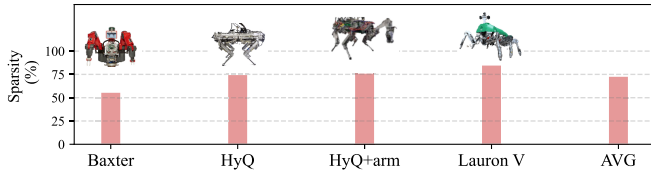


Fig. 2. Sparsity of mass matrix across different robots.

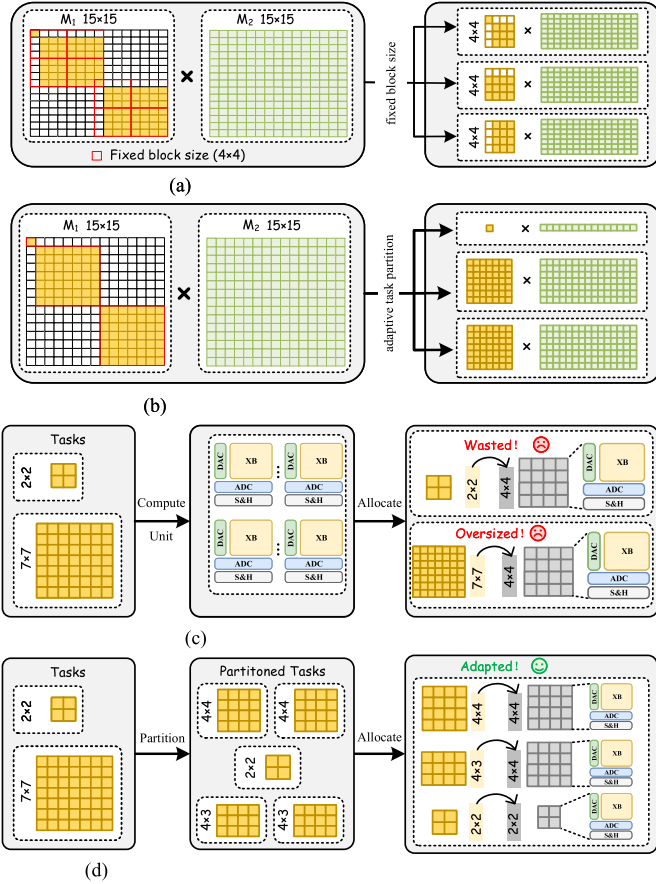


Fig. 3. Illustration of the main idea of RoboPIM. (a) Exploiting sparsity by fixed block size. (b) Exploiting sparsity by adaptive task partition. (c) Problem of data misalignment. (d) Coordinating tasks with crossbars by adaptive task partition.

the sparsity of mass matrix across different robots. Fig. 2 demonstrates the sparsity of mass matrix for four real-world robots. Sparsity of all robots is over 50% and up to 85.21%. Larger robots with more independent limbs tend to exhibit higher sparsity. Failing to leverage this sparsity leads to unnecessary calculations. Previous methods utilize fixed block partitioning to exploit this sparsity. However, the sparsity of the mass matrix exhibits a unique block-diagonal structure, where the sizes of dense diagonal blocks vary significantly (structural heterogeneity), making it impossible for fixed-size schemes to fully exploit this sparsity. As illustrated in Fig. 3(a), when a block diagonal matrix  $M_1$  ( $15 \times 15$ ) multiplies a dense matrix  $M_2$  ( $15 \times 15$ ), the previous method divides the block diagonal matrix into fixed-size blocks to reduce computations involving zero elements.

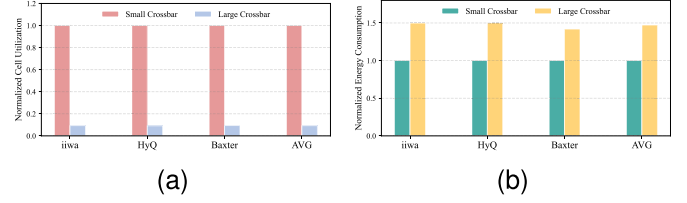


Fig. 4. Normalized (a) cell utilization and (b) energy consumption for robotic tasks between large crossbar and small crossbar.

TABLE I  
FEATURES OF MATRIX MULTIPLICATIONS IN LLM-BASED ROBOTICS APPLICATIONS

|                          | Sparsity | Scale Range                           |
|--------------------------|----------|---------------------------------------|
| $\nabla$ Dynamics [20]   | ✓        | $6 \times 6$ – $15 \times 15$         |
| $\nabla^2$ Dynamics [31] | ✓        | $6 \times 6$ – $15 \times 15$         |
| QKV Generation           |          | $64 \times 64$ – $1536 \times 1536$   |
| Multi-Head Attention     |          | $64 \times 64$ – $1536 \times 1536$   |
| FFN                      |          | $128 \times 128$ – $8960 \times 8960$ |

However, even with the optimal fixed block size ( $4 \times 4$ ) for a specific robot, there are still 18.75% of zero elements remaining in the calculations. Moreover, this block-diagonal pattern is highly variable with robot specifications, necessitating the identification of optimal parameters for different robots, which introduces additional search overhead. Therefore, it is essential to adaptively and fully exploit the varying block-diagonal sparsity patterns to enhance efficiency.

- 2) *Observation 2:* Large disparities in the size of matrix multiplications make ReRAM-based crossbar processing inefficient. Due to the wide range of LLM configurations and robot specifications, matrix multiplication scales with significant disparities, as shown in Table I. Processing them in an integrated ReRAM-based architecture results in inefficiencies, especially for small matrix sizes, as illustrated in Fig. 3(c). When the matrix dimension is much smaller than the crossbar size, only a fraction of the computational units are engaged, while the remaining crossbar cells remain idle. However, the peripheral circuitry—such as analog-to-digital converters (ADCs) and digital-to-analog converters (DACs)—must still operate at full capacity regardless of the matrix size. This results in a disproportionately high energy overhead, where the power consumption of peripheral components does not scale down with reduced computation, rendering small-matrix operations highly inefficient. Specifically, for robotic tasks, our experiments show that large crossbars (e.g.,  $128 \times 128$ ) exhibit  $10.88\times$  lower cell utilization compared to small crossbars (e.g.,  $16 \times 16$ ), as illustrated in Fig. 4(a). This low cell utilization of large crossbars implies that the dominant energy consumption of peripheral circuits is not amortized by effective computations. Instead, it is wasted on invalid data, revealing a critical mismatch between lightweight robotic tasks and the oversized hardware resources. Consequently, these large crossbars consume on average  $1.47\times$  more energy, demonstrating the severe inefficiency.

ciency [see Fig. 4(b)]. This highlights the necessity for an architecture optimized for both computation types.

Based on the above observations, we summarize in Table I the algorithms and operators used in LLM-based robotics applications, highlighting their sparsity and varying scales. These properties make efficient execution on conventional ReRAM-based architectures highly nontrivial. To address these challenges, we propose RoboPIM, a ReRAM-based accelerator tailored for LLM-based robotics applications. RoboPIM features a hybrid architecture that integrates heterogeneous crossbars of different sizes, allowing it to efficiently handle matrix multiplications across various scales by dynamically mapping computations to appropriately sized crossbars as shown in Fig. 3(d). In addition, to efficiently process block-structured sparse matrix multiplications, we introduce an adaptive task partition scheme, which enables the conversion of block-structured sparse matrices into multiple smaller dense submatrices as shown in Fig. 3(b). Through this transformation, RoboPIM can simultaneously and efficiently support both standard dense matrix multiplication and block-structured sparse multiplication, optimizing workload balance and improving crossbar utilization. By combining heterogeneous crossbars with adaptive partition, RoboPIM significantly enhances computational efficiency while mitigating energy overhead associated with peripheral operations.

### III. ROBOPIM ARCHITECTURE

In this section, we provide a detailed description of the RoboPIM architecture. Specifically, we present key components of the architecture, including heterogeneous crossbars of different sizes. We then describe the workflow of RoboPIM from a high-level perspective.

#### A. Architecture

Fig. 5(a) presents the hardware architecture of RoboPIM, an integrated ReRAM-based design that can be embedded in CPUs, GPUs, or FPGA-based accelerators. Matrix multiplications arising from LLM inference and robotics applications, ranging from a few to thousands of dimensions, are efficiently offloaded to the RoboPIM chip and processed in a uniform manner. Meanwhile, an external processor is employed to handle nonmatrix multiplication operations such as power and cross product calculations. A RoboPIM chip consists of external I/O interface that is capable of interacting with off-chip memory through a memory controller. We introduce each component of RoboPIM as follows.

- 1) *Off-Chip Memory*: Off-chip memory includes memory systems such as DRAM that provides a memory access interface. It provides the instructions and original data of matrix multiplication to the RoboPIM chip. Similarly, the results of matrix multiplication tasks generated by the RoboPIM chip can be batch written back to the off-chip memory.
- 2) *Process Element (PE)*: The detail of PE is illustrated in Fig. 5(b). The PE comprises task slicing units, the scheduler, and the compute units (CUs). It receives instructions and matrix data from the I/O interface.

Instructions for SpMM are dispatched to the sparse task slicing unit (STSU), while instructions for dense–dense matrix multiplication (DeMM) are routed to the dense task slicing unit (DTSU). Following task slicing, the scheduler assigns sliced tasks to available CUs. A CU consists of a dispatcher, several data buffers, and a hybrid-sized crossbar array, which is responsible for the efficient execution of matrix multiplication. Matrix data are transferred directly from off-chip memory to the CU via the I/O interface, and the computed matrix blocks are asynchronously written back to the off-chip memory.

- 3) *STSU*: The STSU transforms an SpMM task into several DeMM subtasks through S2D-Slicing described in Section IV-A. As shown in Fig. 5(f), the STSU contains three key components: an SpMM instruction queue and a decoder that receive and parse SpMM instructions, a sparse task queue that caches decoded SpMM tasks, and an S2D-Slicing unit that partitions SpMM tasks into DeMM subtasks. These subtasks are stored in a dense task buffer, encoded into DeMM instructions, and dispatched to DTSU for further slicing.
- 4) *DTSU*: The DTSU converts DeMM tasks into crossbar-adapted subtasks through D2C-Slicing detailed in Section IV-B. As depicted in Fig. 5(d), the DTSU parses DeMM instructions through its dedicated instruction queue and decoder. The decoded tasks, along with their associated metadata (e.g., matrix base addresses and matrix dimensions), are stored in a dense task buffer. Subsequently, a D2C-Slicing unit slices them into smaller subtasks to align with the specifications of the ReRAM crossbar. These generated crossbar-adapted subtasks are ultimately dispatched to the scheduler for execution.
- 5) *Scheduler*: As depicted in Fig. 5(e), the scheduler orchestrates task scheduling through crossbar calculator, dynamic scheduling engine, and CU instruction generator. The crossbar calculator determines crossbar demand for each task according to matrix dimensions and crossbar sizes. The dynamic scheduling engine maintains a max heap for each size of crossbar to track the number of idle crossbars in each CU while executing the scheduling algorithm described in Section IV-C to map tasks to the optimal CU. The CU instruction generator converts scheduling decisions into hardware-executable commands. This architecture enables efficient crossbar management and real-time task allocation.
- 6) *CU*: A CU is responsible for performing assigned matrix multiplication tasks as shown in Fig. 5(c), it includes peripheral circuits, multiple crossbars of different sizes, on-chip buffers, and a dispatcher. Peripheral circuits such as DAC support the crossbars to perform MVM operations. Multiple crossbars of different sizes are responsible for scale-adaptive task execution, where tasks with significant scale disparities are dynamically mapped to appropriate-sized crossbar. This inclusion of a smaller crossbar size is a direct architectural feature to efficiently support the small-scale matrix operations unique to robot dynamics, mitigating the energy over-

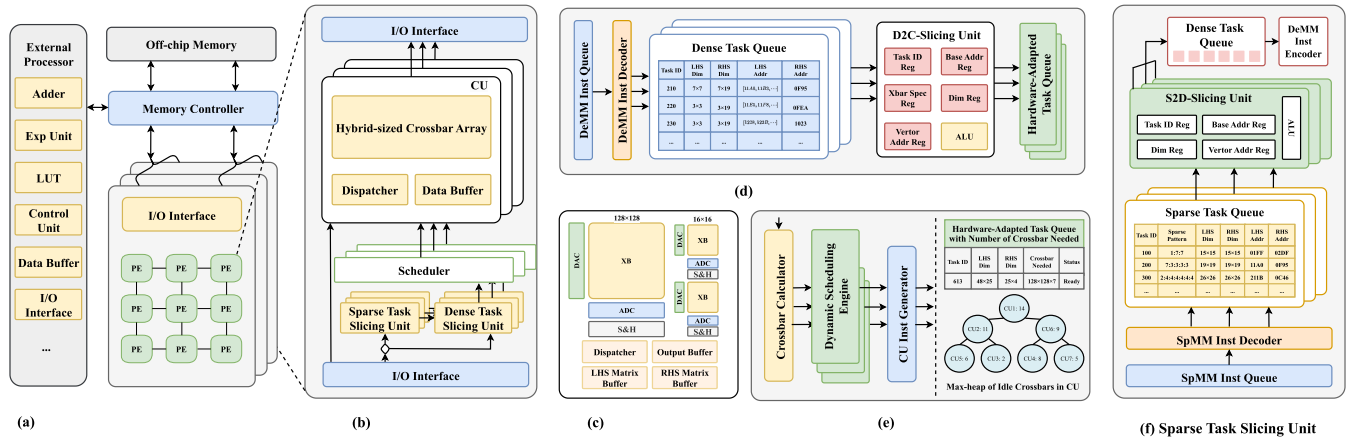


Fig. 5. RoboPIM architecture. (a) Overview of RoboPIM Architecture. (b) PE. (c) CU. (e) Scheduler. (f) STSU.

head observed in oversized LLM-focused crossbars. On-chip buffer consists of right-hand side (RHS) matrix buffer that caches RHS matrix mapped to crossbar, left-hand side (LHS) matrix buffer that caches the input vector of LHS matrix, and output buffer that caches result of a subtask to be batch written back to the off-chip memory. The dispatcher controls the overall data flow in CU, receives the tasks from scheduler, and arranges them to appropriate-sized idle crossbars.

### B. Workflow

Overall, the workflow can be divided into two stages: task slicing stage and dynamic scheduling stage.

At the task slicing stage, the original matrix multiplication tasks are transformed into crossbar-adapted tasks. When a matrix task is offloaded to RoboPIM, it is routed to different task slicing unit according to the matrix characteristics. Sparse matrix multiplication tasks are directed to STSU for sparsity exploitation, while dense matrix multiplication tasks are allocated to DTSU for hardware-adapted partitioning. Moreover, tasks sliced by STSU are further sent to DTSU, since they may still exceed the crossbar capacity. Following DTSU processing, all tasks are properly sized for the crossbar specification, enabling the scheduler to manage them consistently and allowing crossbars to execute them efficiently.

At the dynamic scheduling stage, the properly sized tasks are arranged to ReRAM crossbars for execution. RoboPIM employs crossbars of different sizes to maximize resource efficiency. Larger matrix multiplications are prioritized on larger crossbars, ensuring that the crossbar’s computational capacity is fully utilized, while smaller matrix multiplications are mapped to smaller crossbars to avoid the overhead caused by idle computing cells in oversized crossbars.

To further enhance crossbar utilization, if smaller crossbars are already fully occupied, additional small matrix multiplications may be assigned to larger crossbars to prevent idle resources. Similarly, when larger crossbars reach their full capacity, RoboPIM enables a collaborative crossbar mode, where multiple smaller crossbars are fused together to handle large matrix multiplications through parallel partitioned execution. This adaptive task allocation and crossbar

fusion mechanism allows RoboPIM to achieve high throughput, balanced workload distribution, and improved overall computational efficiency while minimizing energy overhead and idle time.

#### IV. DYNAMIC TASK SLICING AND SCHEDULING

In this section, we introduce the dynamic task slicing scheme, a key strategy designed to enhance the efficiency of matrix multiplications in RoboPIM. This scheme aims to address the challenges posed by block-structured sparsity with slicing sparse matrices into smaller dense submatrices. Furthermore, we describe how the slicing process is further optimized by dynamically adjusting the slicing granularity based on the crossbar sizes. Subsequently, we present the dynamic scheduling method that assigns these sliced tasks to crossbars to maximize hardware utilization. Collectively, these techniques enable RoboPIM to handle both dense and block-structured sparse matrix multiplications with flexibility and efficiency, achieving improved performance and energy efficiency.

### A. Sparsity to Density Task Slicing

The previous method of exploiting sparsity of the mass matrix is to find an optimal block size to slice matrices, excluding the blocks of zero elements. However, this approach cannot fully exploit the sparsity because a fixed block size cannot simultaneously accommodate dense blocks of widely varying sizes. This inflexibility leads to unnecessary computations and suboptimal performance, which can be exacerbated when the robots have asymmetrical limbs (i.e., some limbs have many more links than others).

Therefore, to fully exploit the variable block-structured matrix sparsity, we slice the mass matrix into dense blocks of different sizes according to the robot topology. Specifically, based on the numbers and structure of independent limbs, the topology of robots can be expressed as a topology sequence. For example, Baxter has a head and two limbs with seven sequentially connected links, represented as [1, 7, 7]. This topology sequence indicates that the numbers and sizes of dense blocks in the diagonal of LHS matrix, and thus, the dense blocks can be isolated accordingly. Then the dense

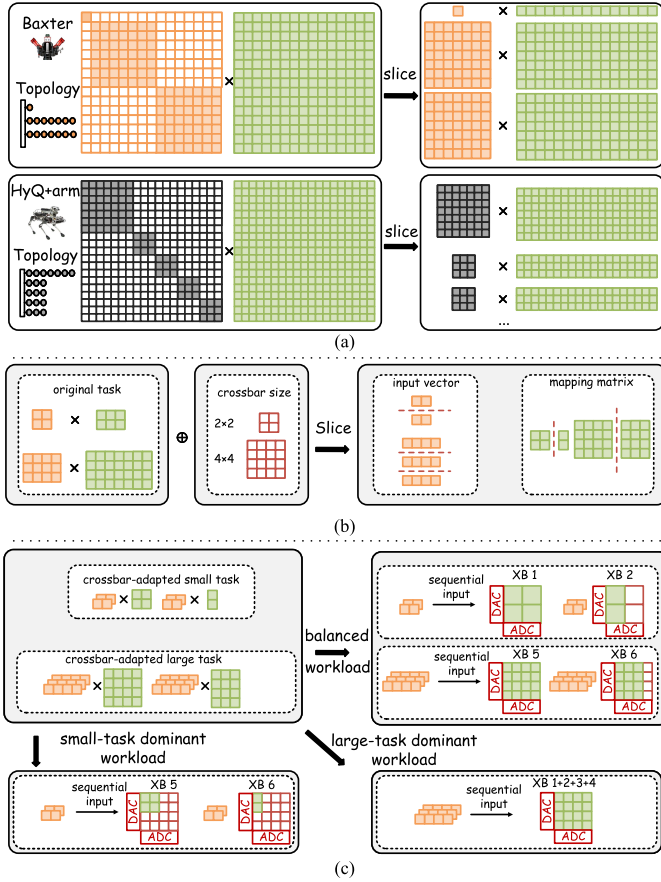


Fig. 6. (a) Sparsity-aware S2D-Slicing. (b) Crossbar-adapted D2C-Slicing. (c) Dynamic task scheduling.

RHS matrix can be sliced into several blocks by rows correspondingly. Consequently, the SpMM tasks are transformed into several DeMM tasks, thus eliminating unnecessary calculations. More general cases are expressed as follows:

$$\mathbf{M} = \mathbf{M}_1 \mathbf{M}_2 = \begin{bmatrix} \mathbf{D}_1 & 0 & \cdots & 0 \\ 0 & \mathbf{D}_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{D}_k \end{bmatrix} \begin{bmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \\ \vdots \\ \mathbf{B}_k \end{bmatrix} = \begin{bmatrix} \mathbf{D}_1 \mathbf{B}_1 \\ \mathbf{D}_2 \mathbf{B}_2 \\ \vdots \\ \mathbf{D}_k \mathbf{B}_k \end{bmatrix}.$$

Given topology sequence  $A = [A_1, A_2, \dots, A_k]$ , the block diagonal matrix  $M_1$  can be sliced into  $k$  dense blocks with block sizes of  $A_1, A_2, \dots, A_k$ . Correspondingly, the RHS matrix  $M_2$  is sliced into  $k$  blocks by rows, denoted as  $B_1, B_2, \dots, B_k$ . Consequently, the SpMM  $M_1 \cdot M_2$  is sliced into several DeMM  $D_i \cdot B_i$  (for  $i = 1, 2, \dots, k$ ). In this way, the sparsity of the mass matrix is fully exploited.

Fig. 6(a) shows the examples of S2D-Slicing. After perceiving the topologies of different robots, the sparse mass matrices are sliced into several dense blocks, while the dense torque gradient matrices are sliced into corresponding blocks that align with the slices of the sparse mass matrix. Consequently, the SpMM tasks are transformed into several DeMM tasks, thus eliminating unnecessary calculations.

In summary, our sparsity-aware S2D-Slicing adaptively and fully exploits the inherent sparsity of the mass matrices across

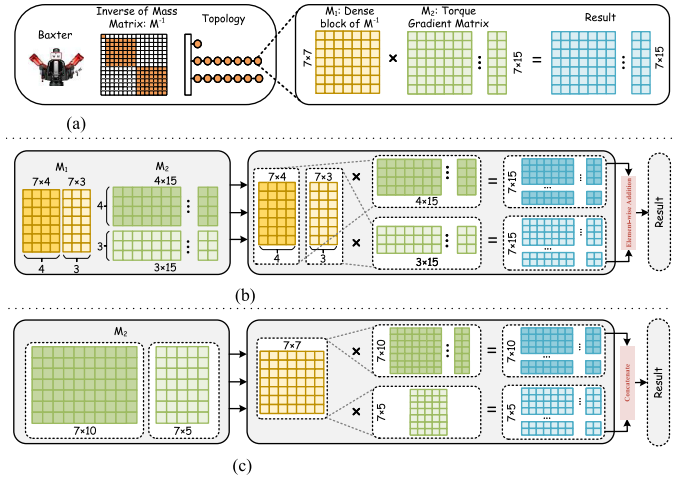


Fig. 7. Comparison between two methods of matrix multiplication slicing. (a) Original matrix multiplication. (b) Horizontal slicing by rows. (c) Vertical slicing by columns.

diverse robots, thereby significantly reducing computational and energy overheads, as well as minimizing latency.

### B. D2C-Slicing

An oversized matrix cannot be directly mapped to the ReRAM crossbar. Thus, it is essential to find a general method for slicing matrix multiplication ranging from a few to thousands of dimension into several subtasks with crossbar-adapted size, thereby enabling uniform processing standard matrix multiplication on RoboPIM.

In particular, there are two ways to slice the oversized tasks: horizontal slicing by rows and vertical slicing by columns. The horizontal and vertical slicing methods we define are for RHS matrices, since RHS matrices are mapped to crossbars, while LHS matrices are split into corresponding vectors as inputs. Take matrix multiplication between inverse of the mass matrix  $M_1$  and torque gradient matrix  $M_2$  for an example. As shown in Fig. 7(a), there is a matrix multiplication task between LHS matrix  $M_1$  and RHS matrix  $M_2$ . If  $M_2$  is oversized, it can be sliced into two blocks. Fig. 7(b) shows the computational flow of horizontal slicing by rows. When we slice a matrix multiplication by rows, two vertical blocks of  $M_1$  multiply two horizontal blocks of  $M_2$ . Due to the inherent rules of matrix multiplication, the results of each subtask need to add together by element to obtain the final result. Thus, additional adders are needed, and the dependencies among subtasks introduce extra synchronization overhead. In contrast, Fig. 7(c) illustrates the computational flow of vertical slicing by columns. The whole matrix  $M_1$  needs to multiply two vertical blocks of  $M_2$ . Finally, the results of the two subtasks are concatenated to produce the overall result. Since the two subtasks are completely independent, this approach preserves the potential for parallelism. Therefore, we select column-wise slicing as our basic partition method.

In addition, how to determine the slicing granularity is another problem. We simply slice the task according to the flooring crossbar size. As shown in Fig. 6(b), if we have

a crossbar of  $2 \times 2$  and  $4 \times 4$ , a three-column RHS matrix is sliced at the granularity of 2, whereas a seven-column matrix aligns with a granularity of 4. Consequently, we employ column-wise slicing to ensure that the RHS matrix width for all matrix multiplication tasks does not exceed the crossbar column size.

The decision to use the crossbar column size as the slicing granularity is primarily based on two considerations. First, slicing matrices by crossbar column size ensures that most subtasks align with the hardware specifications, thereby maximizing hardware utilization. Second, attempting to map matrix multiplication tasks directly to tiled crossbar structures (by assembling multiple crossbars into larger blocks) transforms the problem into a 2-D bin packing problem, an NP-hard issue with no known polynomial-time solutions [32]. In contrast, slicing matrix by crossbar column size reduces the scheduling complexity from two dimensions to one, allowing the needed crossbar number of a task to be determined based solely on the row size of RHS matrix.

### C. Dynamic Task Scheduling

After the processing of S2D-Slicing and D2C-Slicing, the RHS matrix width of each matrix multiplication task does not exceed the column size of the crossbar. Thus, the required number of crossbars for each task can be simply determined by the row size of the RHS matrix. These tasks will be scheduled to crossbars of corresponding size. However, an unbalanced workload may still lead to underutilization of a specific size of crossbar. To address this issue, we design a dynamic scheduling strategy to further improve the hardware utilization. Moreover, different tasks may occupy varying numbers of crossbars with differing durations, necessitating real-time crossbar management. Therefore, we introduce multiple max heaps, each dedicated to a different crossbar size, to track the number of idle crossbars in each CU, facilitating uniform management of ReRAM crossbars and enabling efficient dynamic scheduling.

The scheduling algorithm is divided into two stages, as summarized in Algorithm 2. First, in the initialization stage, sliced matrix multiplication tasks are assigned to two task queues:  $T_1$  is for larger task and  $T_2$  is for smaller. The numbers of idle crossbars with two different sizes in each CU compose two max heaps  $H_1$  and  $H_2$ . Besides, two overall idle crossbar numbers  $N_1$  and  $N_2$  are also maintained to determine whether a task can be scheduled. If the crossbars required by a task in  $T_1$  do not exceed  $N_1$ , this task is offloaded to the CU at the root of the heap, and  $H_1$  is adjusted accordingly. Tasks requiring more crossbars may be iteratively scheduled until the necessary resources are allocated. The initialization stage will continue until there are not enough idle crossbars to accommodate the task. Subsequently, during the dynamic scheduling stage, upon completion of a task in  $T_1$ , the scheduler checks if new tasks in  $T_1$  can be scheduled based on the needed crossbar number and overall idle crossbar number. All scheduling logic of  $T_1$  is applied to  $T_2$ . Moreover, if  $T_2$  is empty and  $T_1$  is not, the idle smaller crossbars will work in a collaborative mode. For instance, if the large crossbar size is  $128 \times 128$  and the small crossbar size is  $32 \times 32$ , four small crossbars will compose a

---

### Algorithm 2 Dynamic Task Scheduling Algorithm

---

```

1: Matrix multiplication task queue  $T_1, T_2$ . Overall idle cross-
   bar number  $N_1, N_2$ . Required crossbar number for task  $t_i$ 
   is  $N_{t_i}$ . Initialize max heap  $H_1, H_2$  of CU capacity and max
   heap root  $R_{H_1}$  and  $R_{H_2}$ .
2: /*Initialization Stage*/
3: while  $T_1$  not empty do
4:    $t_i \leftarrow T_1.\text{peek}()$ 
5:   if  $N_1 \geq N_{t_i}$  then
6:     Allocate  $t_i$  to  $R_{H_1}$ 
7:      $H_1.\text{update}(R_{H_1}, N_{t_i})$  {Adjust max-heap}
8:      $N_1 \leftarrow N_1 - N_{t_i}$ 
9:      $T_1.\text{dequeue}()$ 
10:  else
11:    break
12:  end if
13: end while
14: /* Repeat similar logic for  $T_2$  with  $N_2$  and  $H_2$  */
15: /* Dynamic Scheduling Stage */
16: Once task  $t_j$  is finished
17: if  $t_j \in T_1$  then
18:    $N_1 \leftarrow N_1 + N_{\text{released}}$ 
19:    $H_1.\text{update}(N_{\text{released}})$ 
20:   Schedule  $T_1$  tasks via  $N_1$  and  $H_1$ 
21:   if  $T_1$  empty and  $T_2$  not empty then
22:     allow for scheduling  $T_2$  tasks with  $N_1$  and  $H_1$ 
23:   end if
24: else
25:    $N_2 \leftarrow N_2 + N_{\text{released}}$ 
26:    $H_2.\text{update}(N_{\text{released}})$ 
27:   Schedule  $T_2$  tasks via  $N_2$  and  $H_2$ 
28:   if  $T_2$  empty and  $T_1$  not empty then
29:     allow for scheduling  $T_1$  tasks with  $N_2$  and  $H_2$ 
       {Collaborative mode},
30:   end if
31: end if

```

---

crossbar sized  $32 \times 128$ , align with the column size of large crossbar, and are logically regarded as a large crossbar. Then, a task in  $T_1$  can be scheduled to this logically large crossbar. Similarly, if  $T_1$  is empty and  $T_2$  is not, the task in  $T_2$  can be scheduled to a large crossbar, trading some energy efficiency for performance. Fig. 6(c) demonstrates an example of scheduling. Given multiple dense matrix multiplication tasks, each task will be scheduled by Algorithm 2. Under balanced workloads, large task and small task are assigned to crossbars of matching sizes. In particular, the RHS matrix is mapped to crossbar, while the corresponding rows in the LHS matrix are sequentially input to the crossbar. In a small-task dominant workload situation, a  $2 \times 2$  RHS matrix is also allowed to be mapped to a  $4 \times 4$  crossbar, prioritizing performance over energy efficiency. Similarly, in a large-task dominant workload situation, four  $2 \times 2$  crossbars work in a collaborative mode to process  $4 \times 4$  matrix multiplication. This dynamic scheduling, coupled with our heterogeneous crossbars, enables resource sharing and ensures that the architecture maximizes the utilization for both types of workloads (LLM and robot).

**Algorithm 3** Inverse Dynamics Gradients ( $\nabla$ RNEA) w.r.t.  $u = \{q, \dot{q}\}$ 

```

1: for Link  $i = 1 : N$  do
2:    $\frac{\partial v_i}{\partial u} = {}^iX_{\lambda_i} \frac{\partial v_{\lambda_i}}{\partial u} + \begin{cases} ({}^iX_{\lambda_i} v_{\lambda_i}) \times S_i & u = q \\ S_i & u = \dot{q} \end{cases}$ 
3:    $\frac{\partial a_i}{\partial u} = {}^iX_{\lambda_i} \frac{\partial a_{\lambda_i}}{\partial u} + \frac{\partial v_{\lambda_i}}{\partial u} \times S_i \dot{q}_i + \begin{cases} ({}^iX_{\lambda_i} a_{\lambda_i}) \times S_i & u = q \\ v_i \times S_i & u = \dot{q} \end{cases}$ 
4:    $\frac{\partial f_i}{\partial u} = I_i \frac{\partial a_i}{\partial u} + \frac{\partial v_i}{\partial u} \times I_i v_i + v_i \times I_i \frac{\partial v_i}{\partial u}$ 
5: for Link  $i = N : 1$  do
6:    $\frac{\partial c_i}{\partial u} = S_i^T \frac{\partial f_i}{\partial u}$ 
7:    $\frac{\partial f_{\lambda_i}}{\partial u} = {}^iX_{\lambda_i}^T \frac{\partial f_i}{\partial u} + {}^iX_{\lambda_i}^T (S_i \times f_i)$ 

```

Fig. 8. Fine-grained parallel execution of  $\nabla$ RNEA on ReRAM.

In addition, high write latency and underutilized parallelism are key performance bottlenecks for ReRAM accelerators in iterative algorithms like robot dynamics. To mitigate write overhead, in addition to applying dynamic vectors (e.g., link velocities  $v_i$ ) as input voltages, we further exploit the presence of matrices that remain static during computation. Using the inverse dynamics gradients ( $\nabla$ RNEA) algorithm as an illustrative case (see Fig. 8), link  $\lambda_i$  denotes the parent link of link  $i$ . In motion planning, the trajectory is discretized into a sequence of states, where a single time step refers to one discrete instance (e.g., at time  $t_k$ ). Model constant matrices like spatial inertia ( $I_i \in \mathbb{R}^{6 \times 6}$ ) and motion subspace ( $S_i \in \mathbb{R}^{6 \times 1}$ ) are globally fixed throughout the operation, while the state-dependent spatial coordinate transformation matrices ( $X_i(q) \in \mathbb{R}^{6 \times 6}$ ) are static within the computation of that specific time step. By pre-storing all these matrices onto ReRAM crossbars in a one-time write operation, we eliminate the need for costly, repeated writes. This space-for-time tradeoff effectively minimizes write overhead. Moreover, we employ a fine-grained, term-level parallel scheme to maximize the resource utilization of the ReRAM. As illustrated in Fig. 8, we partition the computation within each recursive step into parallel execution stages (marked as TIKZ IMAGE and TIKZ IMAGE). In each stage, independent additive terms of an expression are mapped to separate ReRAM crossbar arrays for simultaneous execution. This approach fully leverages the ReRAM's inherent parallelism, leading to a significant reduction in the latency of each recursive step.

## V. EVALUATION

### A. Experimental Setup

1) *Simulation*: The fundamental configuration of RoboPIM is summarized in Table II. We develop a cycle-accurate simulation platform for RoboPIM, integrating timing and energy consumption of all components. The simulation framework comprises 32 PEs, with eight CUs per PE. Each CU comprises 32 crossbars sized  $128 \times 128$  and eight crossbars sized  $16 \times 16$ , employing 0T1R-type ReRAM cell. These crossbars have latencies of 29.31 and 50.88 ns for reading and writing, respectively [33]. To mitigate sensing pressure and resistance variability of ReRAM elements, we conservatively employ a 2-bit multilevel cell. Moreover, we configure off-chip memory

TABLE II  
ROBOPIM CONFIGURATION

| Component                              | Param.        | Spec.                                                                         | Power               | Area (mm <sup>2</sup> ) |
|----------------------------------------|---------------|-------------------------------------------------------------------------------|---------------------|-------------------------|
| <b>CU Properties (8 CUs per PE)</b>    |               |                                                                               |                     |                         |
| ADC                                    | resolution    | 8 bits                                                                        | 80 mW               | 0.048                   |
|                                        | number        | 40                                                                            |                     |                         |
| DAC                                    | resolution    | 1 bit                                                                         | 16.5 mW             | 0.0007                  |
|                                        | number        | $32 \times 128, 8 \times 16$                                                  |                     |                         |
| Crossbar Array                         | bits per cell | 2 bits                                                                        | 9.64 mW             | 0.0008                  |
|                                        | configuration | 32 arrays, $128 \times 128$ cells each<br>8 arrays, $16 \times 16$ cells each |                     |                         |
| S&H                                    | number        | $32 \times 128, 8 \times 16$                                                  | $41.25 \mu\text{W}$ | 0.000165                |
| S+A                                    | number        | 40                                                                            | 2 mW                | 0.001                   |
| IR                                     | size          | 8 KB                                                                          | 3.04 mW             | 0.0062                  |
| OR                                     | size          | 1 KB                                                                          | 0.48 mW             | 0.0019                  |
| <b>PE Properties (32 PEs per Chip)</b> |               |                                                                               |                     |                         |
| CUs                                    | number        | 8                                                                             | 893.61 mW           | 0.47012                 |
| CB                                     | size          | 64 KB                                                                         | 17.66 mW            | 0.0086                  |
| IB                                     | size          | 8 KB                                                                          | 1.91 mW             | 0.0054                  |
| OB                                     | size          | 8 KB                                                                          | 1.91 mW             | 0.0054                  |
| SFU                                    | Total         | 8                                                                             | 2.04 mW             | 0.0024                  |
| Scheduler                              | -             | -                                                                             | 58.07 mW            | 0.084                   |
| <b>Chip Properties</b>                 |               |                                                                               |                     |                         |
| PEs                                    | number        | 32                                                                            | 31.21 W             | 18.43                   |
| Controller                             | -             | -                                                                             | 580.41 mW           | 2.65                    |
| Chip Total                             | -             | -                                                                             | 31.79 W             | 21.08                   |

TABLE III  
MODEL CONFIGURATIONS

| Model            | $N_{\text{layer}}$ | $N_{\text{head}}$ | $h_{\text{embedding}}$ | $h_{\text{FFN}}$ |
|------------------|--------------------|-------------------|------------------------|------------------|
| BERT-Small       | 4                  | 8                 | 512                    | 2048             |
| BERT-Base        | 12                 | 12                | 768                    | 3072             |
| GPT-2 Medium     | 24                 | 16                | 1024                   | 4096             |
| GPT-2 Large      | 36                 | 20                | 1280                   | 5120             |
| DeepSeek-R1:1.5B | 28                 | 12                | 1536                   | 8960             |

as high-bandwidth memory (HBM) with 128-GB capacity and 900-GB/s bandwidth. Memory access timing simulations are performed using Ramulator [34], with per-bit energy consumption set to 7 pJ [35]. We use CACTI 7 [36] to model area, power, and access latency of all internal buffers. The resolution of ADCs and DACs is 8 and 2 bit, respectively, and their area and power specifications are based on [37].

2) *Methodology*: We compare RoboPIM with the state-of-the-art design for four typical platform: 1) Baseline: Intel Xeon Gold 5117 CPU with 128-GB DDR4 memory; 2) NVIDIA Tesla V100 GPU with 24-GB HBM memory and 900-GB/s bandwidth; 3) ReBERT [38], a state-of-the-art ReRAM-based accelerator for transformer-based LLMs; and 4) RoboShape [13], a state-of-the-art FPGA accelerator for robotics applications. Additionally, energy consumption of the CPU and GPU is estimated using Intel Product Specifications [39] and the NVIDIA System Management Interface (nvidia-smi) [40], respectively.

3) *Datasets*: We evaluate three representative lightweight transformer-based LLMs, bidirectional encoder representations from transformers (BERT) [41], GPT-2 [42], and DeepSeek-R1 [43] at varying scales for robotics applications. The model configurations are summarized in Table III.  $N_{\text{layer}}$  is the number of transformer blocks.  $N_{\text{head}}$  is the number of attention heads.  $h_{\text{embedding}}$  is the embedding dimension.  $h_{\text{FFN}}$  is the FFN hidden dimension. Furthermore, we consider three distinct robotic platforms, iiwa [44], hydraulically and electrically actuated quadruped (HyQ) [45], and Baxter [46], to evaluate robot dynamics gradients.

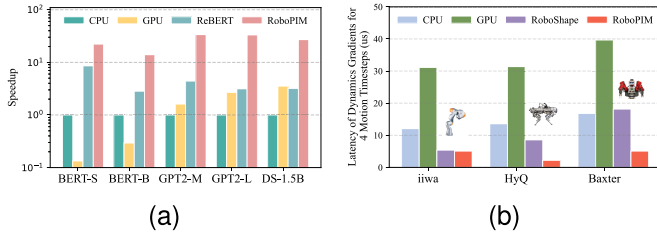


Fig. 9. Performance across (a) different LLMs and (b) different robots. LLM results are normalized to CPU.

### B. Performance and Energy Efficiency

We first compare the performance of RoboPIM on different platforms across various LLMs and robots. Then, we demonstrate the energy efficiency of RoboPIM. Fig. 9 shows the performance of RoboPIM against CPU, GPU, ReBERT, and RoboShape. The details are as follows.

1) *RoboPIM Versus CPU*: Fig. 9(a) shows the performance of RoboPIM on LLM inference compared to CPU. On average, RoboPIM outperforms  $26.02\times$  compared to CPU. This performance advantage stems from the ReRAM-based architecture combined with a dynamic scheduling scheme of RoboPIM, which efficiently performs matrix multiplications in a highly parallel manner. Moreover, as shown in Fig. 9(b), RoboPIM reduces the latency of dynamics gradients by  $3.96\times$  compared to the CPU. This is because parallel technologies such as vectorization for CPU provide limited parallel ability to process these matrix multiplications, while RoboPIM are better at processing them in a fully parallel manner. Moreover, RoboPIM achieves better performance on larger robots such as Baxter, because its workload has a larger potential for parallelism.

2) *RoboPIM Versus GPU*: Fig. 9(a) shows the performance of RoboPIM on LLM inference compared to GPU. RoboPIM outperforms  $51.03\times$  on average compared to GPU. This improvement arises from the fact that D2C-Slicing and dynamic scheduling cooperatively exploit the parallel ability of ReRAM, providing in situ and higher parallel execution of matrix multiplication. Smaller LLMs such as BERT-B gain better improvement than larger LLMs, since they involve less write operations, which is the main time-consuming operation in ReRAM. Moreover, as shown in Fig. 9(b), RoboPIM reduces the latency of dynamics gradients by  $9.43\times$  compared to GPU. The reason behind this is that RoboPIM involves less data movements while adapting to various sparsity patterns in dynamics gradients across different robots.

3) *RoboPIM Versus ReBERT*: Fig. 9(a) demonstrates the performance of RoboPIM on LLM inference compared to ReBERT. Although both systems employ ReRAM-based architecture for LLM acceleration, RoboPIM outperforms by  $6.85\times$  compared to ReBERT. The reasons are twofold. First, RoboPIM employs the dynamic task slicing and scheduling methods to fully exploit the in-matrix parallelism, achieving substantially higher parallelism in matrix multiplication operations. Second, the D2C-Slicing employed by RoboPIM partitions tasks into crossbar-adapted sizes, ensuring that most task sizes align with the hardware specifications, thereby improving hardware utilization and enhancing overall throughput. GPT2-L achieves the best improvement of  $10.58\times$  due

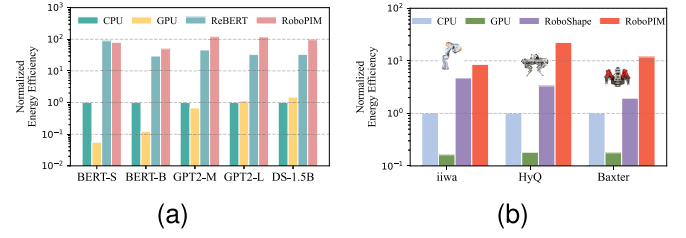


Fig. 10. Energy efficiency across (a) different LLMs and (b) different robots. All results are normalized to CPU.

to its larger number of attention heads, which more fully leverages RoboPIM's parallel processing capabilities.

4) *RoboPIM Versus RoboShape*: Fig. 9(b) demonstrates the performance of RoboPIM on robot dynamics gradients compared to RoboShape. RoboPIM achieves  $2.85\times$  better performance than RoboShape. This significant advantage stems from our architecture's ability to handle the core challenge of motion planning: diverse, topology-dependent sparsity. RoboShape employs a fixed block size to partition matrix tasks, which is inherently inflexible and fails to fully exploit the varying block-structured sparsity patterns generated by different robot topologies. In contrast, RoboPIM's dynamic S2D-Slicing adaptively identifies and isolates the dense sub-blocks based on the specific robot's structure, completely eliminating calculations on zero-element blocks. This adaptive slicing approach, coupled with our fine-grained term-level parallel scheme, ensures maximum computational efficiency. HyQ achieves the best improvements due to its relatively higher potential of parallelism and sparsity to be exploited. Furthermore, these benefits of RoboPIM are scalable to the multilimb robots because RoboPIM is highly parallel and well adapted in matrix multiplication processing.

5) *Energy Efficiency*: Fig. 10(a) shows the energy efficiency of RoboPIM compared to CPU, GPU, and ReBERT in LLM inference. RoboPIM consumes  $93.36\times$  and  $435.99\times$  less energy on average compared to CPU and GPU, respectively. This is because the ReRAM crossbar naturally performs matrix multiplications in a high-parallel and energy-efficient way while involving less data movements compared to conventional architectures. RoboPIM also achieves an energy efficiency improvement of  $2.35\times$  over ReBERT. This efficiency is primarily attributed to our task slicing and scheduling method that optimizes the hardware utilization. In addition, as illustrated in Fig. 10(b), RoboPIM saves  $14.22\times$ ,  $80.56\times$ , and  $4.87\times$  energy compared to CPU, GPU, and RoboShape in robot dynamics gradients. The reason behind this is that RoboPIM processes matrix multiplications through in situ computing while fully exploiting the sparsity in dynamics gradients, thereby eliminating unnecessary data transfers and calculations.

6) *Energy Efficiency Analysis*: To substantiate the efficiency benefits of our proposed architecture, we perform a detailed energy breakdown across different robotic tasks, as shown in Fig. 11. Although the ADC dominates the area in both configurations, the small crossbar demonstrates  $1.47\times$  higher energy efficiency compared to the large crossbar. The reasons are twofold. First, the large crossbar suffers from prolonged peripheral activation. Specifically, the ADC requires 128 sampling cycles to complete a row readout, whereas

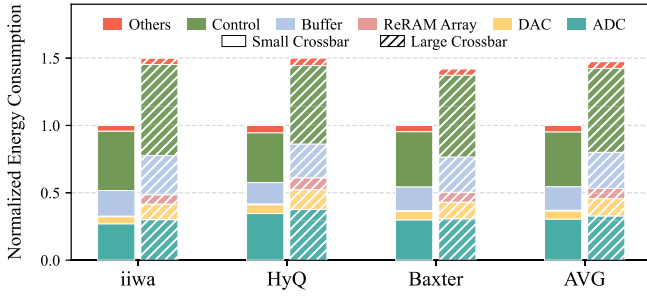


Fig. 11. Normalized energy breakdown comparing small ( $16 \times 16$ ) and large ( $128 \times 128$ ) crossbars across different robotic tasks.

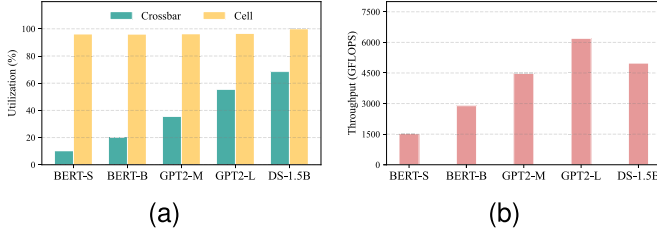


Fig. 12. (a) Crossbar utilization and cell utilization and (b) throughput in different LLMs.

the small crossbar requires only 16. During these additional cycles, the shared peripheral circuits (control and buffer in Fig. 11) must also remain powered on to sustain instruction scheduling and data I/O. Second, the large crossbar incurs higher intrinsic overheads from driving more DACs and handling larger array parasitics, which result in wasted energy when the crossbar is significantly underutilized.

7) *Hardware Utilization and Throughput*: We further evaluate the efficiency of our dynamic task slicing and scheduling strategy by hardware utilization and throughput. Fig. 12(a) illustrates the crossbar and cell utilization across different LLMs. On average, RoboPIM achieves excellent crossbar and cell utilization of 38.06% and 97.03%, respectively. Crossbar utilization increases as the model size increases, while cell utilization remains consistently high across all models, indicating its scalability. The reason behind this is our heterogeneous crossbars of different sizes combining with the dynamic task slicing and scheduling strategy, which significantly mitigates data misalignment and enables efficient execution of matrix multiplications ranging from a few to thousand of dimensions on ReRAM. Fig. 12(b) demonstrates the throughput on different LLMs. RoboPIM attains 4013.45 giga floating point operations per second (GFLOPS) on average. This is because our task slicing and scheduling strategy cooperately fully exploits the parallel capability of ReRAM. Larger models such as DS-1.5B achieve higher throughput, because its larger workload is able to exploit the higher intrinsic parallelism of ReRAM crossbar.

### C. Scalability and Sensitivity Analysis

To demonstrate the impact of different configurations of RoboPIM, we further evaluate the performance under various crossbar sizes and read-to-write ratios.

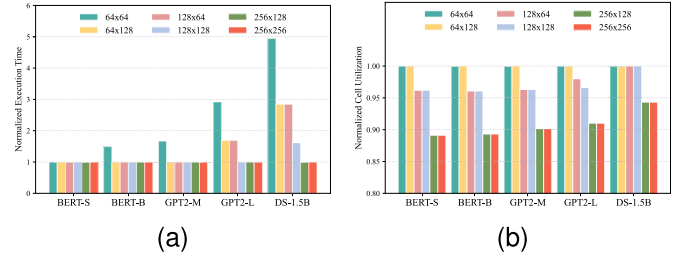


Fig. 13. (a) Normalized execution time. (b) Normalized cell utilization of various crossbar sizes in different LLMs.

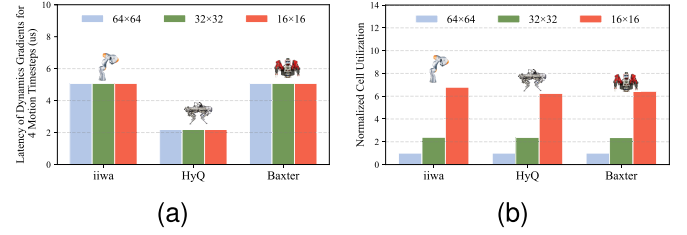


Fig. 14. (a) Execution time. (b) Normalized cell utilization of various crossbar sizes across different robots.

1) *Large Crossbar Size*: Large crossbar in RoboPIM is employed to execute large-scale matrix multiplication in a highly parallel manner. Fig. 13(a) shows the normalized execution time of RoboPIM with various crossbar sizes in different LLMs. For small models such as BERT-S,  $64 \times 64$  is sufficient to fully exploit the parallel potential of the workload. For large models such as DS-1.5B, the execution time scales inversely and almost linearly with crossbar size, demonstrating excellent scalability on crossbar size. Moreover, crossbars of the same size but different shapes, such as  $64 \times 128$  and  $128 \times 64$ , exhibit similar performance, showcasing good adaptability of RoboPIM. These results are attributed to the task slicing and scheduling strategy employed by RoboPIM, which effectively coordinates matrices ranging from a few to thousands of dimensions with crossbar sizes, ensuring efficient crossbar utilization and excellent adaptability in matrix multiplications. Fig. 13(b) further illustrates the normalized cell utilization with varying crossbar sizes in different LLMs. The cell utilization of RoboPIM remains robust with various workloads and crossbar sizes. BERT-S exhibits the most significant decrease in cell utilization, with a reduction of 10.89% when scaling from a  $64 \times 64$  crossbar to a  $256 \times 256$  configuration, as its relatively small workload is insufficient to fully utilize the larger crossbars.

2) *Small Crossbar Size*: Small crossbar in RoboPIM is employed to process light workloads such as robot dynamics gradients in an energy-efficient manner. Fig. 14(a) demonstrates the latency of dynamics gradients across different robots on various sizes of crossbars. Every crossbar size achieves nearly identical performance, since our dynamic task slicing and scheduling strategy can adaptively coordinate matrix dimensions with crossbar sizes, ensuring efficient processing of matrix multiplications. Fig. 14(b) shows the normalized cell utilization under different crossbar size configurations. Overall, smaller sizes of crossbar achieve higher cell utilization

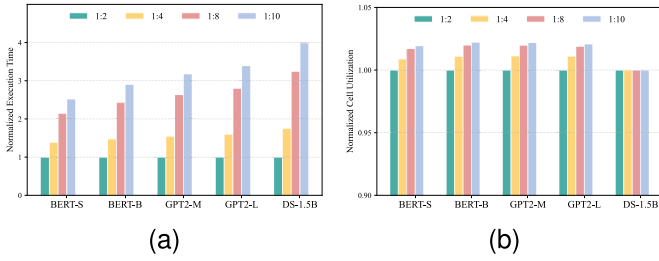


Fig. 15. (a) Normalized execution time. (b) Normalized cell utilization of various read-to-write ratios in different LLMs.

for all robotic workloads. Crossbar of  $16 \times 16$  demonstrates  $6.48 \times$  higher cell utilization compared to  $64 \times 64$  configuration, leaving significantly fewer idle crossbar cells. These results suggest prioritizing smaller crossbars to enhance energy efficiency. Given that excessively small crossbar may lead to extra peripheral circuit and communication overhead, we select  $16 \times 16$  as the small crossbar size, making a balanced tradeoff between energy efficiency and peripheral complexity.

3) *Read-to-Write Ratio*: We evaluate the performance of RoboPIM with various read-to-write ratios. Fig. 15(a) illustrates the performance with the read-to-write ratios of 1:2, 1:4, 1:8, and 1:10. Overall, the performance degrades as the write cost increases. However, the performance degradation is much less than the increase in write cost. This is because the write cost does not significantly impact the high parallelism of RoboPIM with our adaptive task slicing and dynamic scheduling scheme. Fig. 15(b) further demonstrates the normalized cell utilization across various read-to-write ratios. The cell utilization remains almost unchanged from 1:2 to 1:10, showcasing excellent stability across different read-to-write ratios.

## VI. RELATED WORK

### A. Robotics Application Accelerator

Robotics applications demand real-time, energy-efficient processing. General-purpose approaches like [47] leverage GPU parallelism for motion planning, while Tartan [12] provides CPU architectural support, but these fail to meet energy efficiency requirements for resource-constrained systems. Domain-specific accelerators [15], [48] leverage spatial locality and sparsity for motion planning, demonstrating improved efficiency but are limited by fixed block sizes for sparsity and narrow support for varying matrix scales. Prior dynamic partitioning techniques for ReRAM [49], [50] focus almost exclusively on single-scale, dense, or unstructured-sparse workloads typical of deep learning, failing to address the dual heterogeneity of integrated LLM-robotics systems. RoboPIM addresses this gap with a two-stage dynamic task slicing (S2D-Slicing and D2C-Slicing) and heterogeneous crossbar architecture, enabling adaptive exploitation of topology-dependent sparsity and efficient matrix processing across extreme dimensional range.

### B. Transformer-Based LLM Accelerator

Hardware acceleration for LLM inference has emerged as a critical research focus due to its escalating

computational demands. IANUS [11] and NeuPIM [10] combine NPU parallelism with PIM bandwidth, but remain unsuitable for robotic scenarios due to energy demands. Other approaches optimize KV Cache [51], [52] or leverage sparse attention [38], [53] but fail to address structural and scale heterogeneity in LLM-based robotics applications. RoboPIM employs dynamic matrix slicing to adaptively accelerate matrix multiplication with varying sparsity patterns, utilizing heterogeneous crossbars and dynamic scheduling to optimize resource utilization across scale disparities.

## VII. CONCLUSION

In this article, we present RoboPIM, the first integrated ReRAM-based architecture designed for LLM-based robotics applications. RoboPIM addresses the challenges of structural and scale heterogeneity in matrix multiplication on ReRAM, achieving significant improvements in both performance and energy efficiency. To fully adapt to these two features for LLM-based robotics applications, at the hardware level, RoboPIM incorporates heterogeneous crossbars of different sizes and a dynamic scheduling strategy to support uniform processing of matrix multiplication, enhancing the hardware utilization and parallelism. At the software level, RoboPIM develops two types of task slicing, S2D-Slicing and D2C-Slicing. The S2D-Slicing transforms block-structured sparse matrix multiplication into multiple dense submatrix multiplication, allowing for adaptive exploitation of varying sparsity. Meanwhile, the D2C-Slicing partitions dense matrix multiplication across varying scales (from a few to thousands of dimensions) into crossbar-adapted sizes for hardware-efficient execution on the ReRAM-based architecture. Extensive evaluation demonstrates that RoboPIM outperforms CPU, GPU, and state-of-the-art domain-specific accelerators in both performance and energy efficiency.

## REFERENCES

- [1] X. Gao and Z. Wen, "Task-oriented robotic grasping for intelligent manufacturing," in *Proc. 3rd Int. Symp. Artif. Intell. Intell. Manuf. (AIIM)*, Oct. 2023, pp. 101–104.
- [2] C. Zheng et al., "Knowledge-based engineering approach for defining robotic manufacturing system architectures," *Int. J. Prod. Res.*, vol. 61, no. 5, pp. 1436–1454, Mar. 2023.
- [3] M. Yip et al., "Artificial intelligence meets medical robotics," *Science*, vol. 381, no. 6654, pp. 141–146, Jul. 2023.
- [4] Z. Yan, Z. Bai, T. Mitra, and W.-F. Wong, "SparrowSNN: A hardware/software co-design for energy efficient ECG classification," 2024, *arXiv:2406.06543*.
- [5] W. Xiao, X. Li, L. Hu, Y. Hao, and M. Chen, "DTSNet: Dynamic transformer slimming for efficient vision recognition," *IEEE Trans. Multimedia*, pp. 1–12, 2025, doi: [10.1109/TMM.2025.3607796](https://doi.org/10.1109/TMM.2025.3607796).
- [6] J. Shang, X. Li, K. Kahatapitiya, Y.-C. Lee, and M. S. Ryoo, "StARformer: Transformer with state-action-reward representations for robot learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 12862–12877, Nov. 2023.
- [7] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [8] C. Liu et al., "HeterRAG: Heterogeneous processing-in-memory acceleration for retrieval-augmented generation," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, Jun. 2025, pp. 884–898.
- [9] B. Gao, Z. He, Y. Yao, Z. L. Lou, Z. Zhou, and W.-F. Wong, "Online context caching for distributed large language models serving," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2025, pp. 1–10.
- [10] G. Heo et al., "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2024, pp. 722–737.

- [11] M. Seo et al., "IANUS: Integrated accelerator based on NPU-PIM unified memory system," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, vol. 3, Apr. 2024, pp. 545–560.
- [12] M. Bakhshalipour and P. B. Gibbons, "Tartan: Microarchitecting a robotic processor," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 548–565.
- [13] S. M. Neuman, R. Ghosal, T. Bourgeat, B. Plancher, and V. J. Reddi, "RoboShape: Using topology patterns to scalably and flexibly deploy accelerators across robots," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–13.
- [14] L. Huang, Y. Gong, Y. Sui, X. Zang, and B. Yuan, "MOPED: Efficient motion planning engine with flexible dimension support," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 483–497.
- [15] D. Shah and T. M. Aamodt, "Collision prediction for robotics accelerators," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 566–581.
- [16] H. Li, D. Chen, and T. Mitra, "SPLIM: Bridging the gap between unstructured SpGEMM and structured in-situ computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 6, pp. 2412–2423, Jun. 2025.
- [17] J. Sun, F. Liu, Y. Zhang, L. Jiang, and R. Yang, "RTSA: An RRAM-TCAM based in-memory-search accelerator for sub-100  $\mu$ s collision detection," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–2.
- [18] W. Xiao et al., "ChainPIM: A ReRAM-based processing-in-memory accelerator for HGNNs via chain structure," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2516–2529, Jul. 2025.
- [19] C. Wolters, X. Yang, U. Schlichtmann, and T. Suzumura, "Memory is all you need: An overview of compute-in-memory architectures for accelerating large language model inference," 2024, *arXiv:2406.08413*.
- [20] J. Carpentier and N. Mansard, "Analytical derivatives of rigid body dynamics algorithms," in *Proc. Robot., Sci. Syst. XIV*, Jun. 2018, pp. 1–9, doi: [10.15607/RSS.2018.XIV.038](https://doi.org/10.15607/RSS.2018.XIV.038).
- [21] R. Featherstone, *Rigid Body Dynamics Algorithms*. Cham, Switzerland: Springer, 2014.
- [22] A. Roennau, G. Heppner, M. Nowicki, and R. Dillmann, "LAURON V: A versatile six-legged walking robot with advanced maneuverability," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatronics*, Jul. 2014, pp. 82–87.
- [23] D. Chen et al., "A general offloading approach for near-DRAM processing-in-memory architectures," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, May 2022, pp. 246–257.
- [24] D. Chen et al., "MetaNMP: Leveraging Cartesian-like product to accelerate HGNNs with near-memory processing," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–13.
- [25] H. Jin et al., "Accelerating graph convolutional networks through a PIM-accelerated approach," *IEEE Trans. Comput.*, vol. 72, no. 9, pp. 2628–2640, Sep. 2023.
- [26] D. Chen et al., "GraphFly: Efficient asynchronous streaming graphs processing via dependency-flow," in *Proc. SC Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2022, pp. 1–14.
- [27] C. Liu et al., "SeIM: In-memory acceleration for approximate nearest neighbor search," in *Proc. 62nd ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2025, pp. 1–7.
- [28] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024.
- [29] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Aug. 2019, doi: [10.1109/MSSC.2019.2922889](https://doi.org/10.1109/MSSC.2019.2922889).
- [30] S. Yu, "Neuro-inspired computing with emerging nonvolatile memories," *Proc. IEEE*, vol. 106, no. 2, pp. 260–285, Feb. 2018.
- [31] J. N. Nganga and P. M. Wensing, "Accelerating second-order differential dynamic programming for rigid-body systems," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7659–7666, Oct. 2021.
- [32] E. G. Coffman Jr., M. R. Garey, and D. S. Johnson, "Approximation algorithms for bin-packing—An updated survey," in *Algorithm Design for Computer System Design*. Cham, Switzerland: Springer, 1984, pp. 49–106.
- [33] D. Niu, C. Xu, N. Muralimanohar, N. P. Jouppi, and Y. Xie, "Design of cross-point metal-oxide ReRAM emphasizing reliability and cost," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2013, pp. 17–23.
- [34] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, pp. 45–49, Jan. 2016.
- [35] M. O'Connor, "Highlights of the high-bandwidth memory (HBM) standard," in *Proc. Memory Forum Workshop (MFW)*, Minneapolis, MN, USA, Jun. 2014, pp. 1–25.
- [36] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, pp. 1–25, Jun. 2017.
- [37] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 14–26, 2016.
- [38] M. Kang, H. Shin, and L.-S. Kim, "A framework for accelerating transformer-based language model on ReRAM-based architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 3026–3039, Sep. 2022.
- [39] Intel. *Intel Xeon Gold 5117 Processor (19.25m Cache, 2.00 GHz) Specifications*. Accessed: Feb. 8, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/122460/intel-xeon-gold-5117-processor-19-25m-cache-2-00-ghz/specifications.html>
- [40] Nvidia. (2012). *NVIDIA System Management Interface*. [Online]. Available: <https://developer.nvidia.com/nvidia-system-management-interface>
- [41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol. (Long Short Papers)*, vol. 1, 2019, pp. 4171–4186.
- [42] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [43] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
- [44] K. AG. (2024). *LBR IIWA*. Accessed: 2024. [Online]. Available: <https://www.kuka.com/products/roboticssystems/industrial-robots/lbr-iiwa>
- [45] C. Semini, N. G. Tsagarakis, E. Guglielmino, M. Focchi, F. Cannella, and D. G. Caldwell, "Design of HYQ—A hydraulically and electrically actuated quadruped robot," *Proc. Inst. Mech. Eng., I, J. Syst. Control Eng.*, vol. 225, no. 6, pp. 831–849, 2011.
- [46] C. Fitzgerald, "Developing baxter," in *Proc. IEEE Conf. Technol. Practical Robot Appl. (TePRA)*, Apr. 2013, pp. 1–6.
- [47] J. Liu, G. Xiao, F. Wu, X. Liao, and K. Li, "AAPP: An accelerative and adaptive path planner for robots on GPU," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2336–2349, Aug. 2023.
- [48] M. Bakhshalipour, S. B. Ehsani, M. Qadri, D. Guri, M. Likhachev, and P. B. Gibbons, "RACOD: Algorithm/hardware co-design for mobile robot path planning," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, Jun. 2022, pp. 597–609.
- [49] H. Li, Z. Li, Z. Bai, and T. Mitra, "ASADI: Accelerating sparse attention using diagonal-based in-situ computing," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Mar. 2024, pp. 774–787.
- [50] W. Xiao, C. Shi, M. Chen, Z. Liu, M. Chen, and H. H. Song, "GraphEdge: Dynamic graph partition and task scheduling for GNNs computing in edge network," *Inf. Fusion*, vol. 124, Dec. 2025, Art. no. 103329.
- [51] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.
- [52] Y. Zhao, D. Wu, and J. Wang, "ALISA: Accelerating large language model inference via sparsity-aware KV caching," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 1005–1017.
- [53] H. Li, D. Chen, and T. Mitra, "SADIMM: Accelerating sparse attention using DIMM-based near-memory processing," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 542–554, Feb. 2025.



**Wenjing Xiao** (Member, IEEE) received the Ph.D. degree from the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China, in 2023.

She is a Research Assistant Professor with the School of Computer, Electronic and Information, Guangxi University, Nanning, China. Her research interests include efficient artificial intelligence, edge intelligence, on-device artificial intelligence, the Internet of Things, and wireless networks.



**Jianyu Wang** received the B.E. degree in mechanical engineering from the School of Mechanical Engineering, Jiangnan University, Wuxi, China, in 2022. He is currently pursuing the M.E. degree with the School of Computer, Electronics and Information, Guangxi University, Nanning, China.

His research interests include in-memory computing and graph neural networks.



**Mohsen Guizani** (Fellow, IEEE) received the B.S. degree (Hons.) and the M.S. and Ph.D. degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA, in 1985, 1987, and 1990, respectively.

Previously, he worked in different institutions in USA. He is currently a Professor of machine learning and the Associate Provost at Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates. His research interests include applied machine learning and artificial intelligence, the Internet of Things (IoT), intelligent autonomous systems, smart city, and cybersecurity.



**Dan Chen** received the Ph.D. degree from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2024.

He is now working toward a Research Fellow at the National University of Singapore, Singapore. His research interests focus on graph processing and processing-in-memory.



**Min Chen** (Fellow, IEEE) is a Full Professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China. He is also the Director of the Embedded and Pervasive Computing (EPIC) Lab, Huazhong University of Science and Technology, Wuhan, China.

Prof. Chen is the Founding Chair of the IEEE Computer Society Special Technical Communities on Big Data. His Google Scholar Citations reached more than 45 800 with an H-index of 98. His top paper was cited more than 4750 times. He was selected as a Highly Cited Researcher from 2018 to 2023.



**Huize Li** received the Ph.D. degree from the School of Computer Science and Technology, Huazhong University of Science and Technology (HUST), Wuhan, China, in 2022.

He is now working as a Post-Doctoral Researcher at the Department of Electrical and Computer Engineering, University of Central Florida (UCF), Orlando, FL, USA. His current research interests include computer architecture, emerging nonvolatile memory, and processing in memory.



**Thomas Wu** received the Ph.D. degree in electrical engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 1999.

He is currently a Distinguished Professor and the Dean of the School of Electrical Engineering, China-ASEAN Energy Institute, and the Artificial Intelligence Research Institute, Guangxi University, Nanning, China.